

C# & .NET Interview Differences Decoded

90 must-know "X vs Y" concepts across C# Fundamentals, OOP, ASP.NET Core, Entity Framework Core & SQL Server — explained simply, with tables and code, so you walk into your next interview fully prepared.

90

QUESTIONS

5

CORE TOPICS

30+

CODE SAMPLES

C# Fundamentals

OOP & Advanced C#

ASP.NET Core

Entity Framework Core

SQL Server



Created by Kaushal Rao

Software Engineer | Tech Mentor | Tech Enthusiast

[in Kaushalg-rao](#) [Kaushal-Rao14](#)

Table of Contents

90 interview-ready comparisons, grouped by topic area.

01 C# Fundamentals

Q1–Q20

- | | |
|----------------------------------|--|
| Q1. Class vs Struct | Q11. == vs .Equals() |
| Q2. Class vs Interface | Q12. Equals() vs ReferenceEquals() |
| Q3. Abstract Class vs Interface | Q13. String vs StringBuilder |
| Q4. Field vs Property | Q14. String vs StringBuilder vs StringBuffer |
| Q5. Property vs Method | Q15. var vs dynamic vs object |
| Q6. const vs readonly | Q16. is vs as |
| Q7. readonly vs static readonly | Q17. Method Overloading vs Method Overriding |
| Q8. ref vs out vs in | Q18. Virtual vs Override vs New |
| Q9. Value Type vs Reference Type | Q19. Static Class vs Singleton |
| Q10. Stack vs Heap | Q20. Early Binding vs Late Binding |

02 OOP & Advanced C#

Q21–Q40

- | | |
|---|--|
| Q21. Encapsulation vs Abstraction | Q31. Delegate vs Event |
| Q22. Inheritance vs Composition | Q32. Delegate vs Func vs Action |
| Q23. Interface vs Abstract Class (when to choose) | Q33. IEnumerable vs ICollection vs IList |
| Q24. Association vs Aggregation vs Composition | Q34. IEnumerable vs IQueryable |
| Q25. Shallow Copy vs Deep Copy | Q35. List<T> vs Array |
| Q26. Managed Code vs Unmanaged Code | Q36. Dictionary vs Hashtable |
| Q27. Garbage Collection vs IDisposable | Q37. Thread vs Task |
| Q28. Dispose vs Finalize | Q38. Task vs async/await |
| Q29. Exception vs Error | Q39. Parallelism vs Concurrency |
| Q30. Throw vs Throw ex | Q40. Synchronous vs Asynchronous Programming |

03 ASP.NET Core

Q41–Q60

- | | |
|--|---|
| Q41. ASP.NET MVC vs ASP.NET Core | Q51. Dependency Injection vs Service Locator |
| Q42. MVC vs Web API | Q52. Constructor Injection vs Method Injection |
| Q43. Controller vs API Controller | Q53. IActionResult vs ActionResult vs ActionResult<T> |
| Q44. Middleware vs Filter | Q54. HTTP GET vs POST |
| Q45. Authentication vs Authorization | Q55. PUT vs PATCH |
| Q46. JWT vs Cookie Authentication | Q56. POST vs PUT |
| Q47. Session vs Cookies | Q57. HTTP 401 vs 403 |
| Q48. ViewData vs ViewBag vs TempData | Q58. HTTP 404 vs 400 |
| Q49. Action Filter vs Authorization Filter | Q59. HTTP 500 vs 503 |

04 Entity Framework Core

Q61–Q74

- Q61. EF Core vs ADO.NET
- Q62. EF Core vs Dapper
- Q63. Code First vs Database First
- Q64. Tracking vs NoTracking
- Q65. Find() vs FirstOrDefault()
- Q66. First() vs FirstOrDefault()
- Q67. Single() vs SingleOrDefault()
- Q68. Where() vs Find()
- Q69. Eager Loading vs Lazy Loading vs Explicit Loading
- Q70. IEnumerable vs IQueryable in EF Core
- Q71. Migration vs Database Update
- Q72. DbContext vs DbSet
- Q73. Stored Procedure vs LINQ Query
- Q74. SaveChanges vs SaveChangesAsync

05 SQL Server

Q75–Q90

- Q75. DELETE vs TRUNCATE vs DROP
- Q76. WHERE vs HAVING
- Q77. INNER JOIN vs LEFT JOIN
- Q78. UNION vs UNION ALL
- Q79. Primary Key vs Unique Key
- Q80. Primary Key vs Foreign Key
- Q81. Clustered vs Non-Clustered Index
- Q82. CHAR vs VARCHAR
- Q83. VARCHAR vs NVARCHAR
- Q84. Stored Procedure vs Function
- Q85. View vs Stored Procedure
- Q86. CTE vs Temporary Table
- Q87. Temporary Table vs Table Variable
- Q88. DELETE vs MERGE
- Q89. Normalization vs Denormalization
- Q90. Subquery vs CTE

01

C# Fundamentals

Core language building blocks every C# developer must master.

Q1 Class vs Struct

- Class is a **reference type**; Struct is a **value type**.
- Class instances live on the heap; struct values usually live on the stack (or inline inside the containing object).
- Class supports inheritance from another class; struct cannot inherit a class (only interfaces).
- Assigning a class variable copies the reference; assigning a struct copies the entire value.
- Use struct for small, immutable data (Point, DateTime); use class for complex objects with identity and behaviour.

Aspect	Class	Struct
Type category	Reference type	Value type
Memory	Heap	Stack / inline
Inheritance	Supported	Not supported (interfaces only)
Null value	Allowed	Not allowed (unless Nullable<T>)
Copy behaviour	Copies reference	Copies actual data

```
class Person { public string Name; } // reference type
struct Point { public int X, Y; } // value type

var p1 = new Point { X = 1, Y = 2 };
var p2 = p1; // copies the VALUE
p2.X = 100; // p1.X is still 1
```

Q2 Class vs Interface

- A class can provide full implementation; an interface (pre C# 8) only declares a contract.
- A class can inherit only ONE base class, but can implement MULTIPLE interfaces.
- A class can have fields and constructors; an interface cannot have instance fields or constructors.
- Interface answers 'what a type can do'; class answers 'how it does it'.

Aspect	Class	Interface
Multiple inheritance	No (single base class)	Yes (many interfaces)
Implementation	Full method bodies	Only default methods (C# 8+)
Fields	Allowed	Not allowed
Constructors	Allowed	Not allowed

```
interface IShape { double Area(); }

class Circle : IShape
{
    public double Radius;
    public double Area() => Math.PI * Radius * Radius;
}
```

Q3 Abstract Class vs Interface

- Abstract class can mix abstract AND concrete (implemented) members; interface historically holds only contracts (C# 8+ allows default bodies).
- A class can extend only one abstract class but can implement many interfaces.
- Abstract class can hold fields, constructors and access modifiers; interface traditionally cannot.
- Prefer abstract class for closely related types that share common code/state; prefer interface for unrelated types that just share a capability.

Aspect	Abstract Class	Interface
Members	Abstract + concrete	Mostly abstract (default methods in C#8+)
Multiple inheritance	No	Yes
Constructors / fields	Allowed	Not allowed
Best for	Shared base behaviour	Shared capability/contract

Q4 Field vs Property

- A field is a plain variable declared directly inside a class.
- A property wraps a (usually private) field with get/set accessors, allowing validation and encapsulation.
- Interfaces can declare properties, but never fields.
- Properties support data-binding (WPF, ASP.NET); raw fields do not.

```
private string _name;           // field

public string Name              // property
{
    get => _name;
    set => _name = value ?? throw new ArgumentNullException();
}

public int Age { get; set; }    // auto-property
```

Q5 Property vs Method

- Property represents **state** (an attribute of the object); Method represents **behaviour** (an action it performs).
- A property should be cheap and side-effect-free, like reading a field; a method can be expensive and can have side effects.
- A property is accessed like a field (obj.Name); a method is invoked with parentheses (obj.GetName()).
- Properties can't take parameters (except indexers); methods can accept any number of parameters.

Q6 const vs readonly

- **const** value must be known and assigned at compile time; **readonly** can be assigned at declaration OR inside a constructor.
- const is implicitly static and shared by the type; readonly can be per-instance or static.
- const only works with primitive types, strings and enums; readonly works with any type, including objects and arrays.
- const values get baked into the calling assembly at compile time — changing them later needs a full recompile of dependents.

```
public const double Pi = 3.14159;           // fixed at compile time

public readonly DateTime Created;           // set once, at runtime
public MyClass() { Created = DateTime.Now; }
```

Q7 readonly vs static readonly

- Instance **readonly**: every object can hold its own value, typically set inside the constructor.
- **static readonly**: one single value shared by ALL instances of the class, can be set in a static constructor.
- static readonly is initialised once, the first time the type is used; instance readonly is initialised per object.
- Use static readonly for a class-wide constant-like value that isn't known until runtime (e.g. read from config).

Q8 ref vs out vs in

- **ref**: passes by reference; the variable must already be initialised before the call; can be read and modified.
- **out**: passes by reference; the variable does NOT need to be initialised beforehand, but MUST be assigned inside the method.
- **in**: passes by reference but is READ-ONLY inside the method — avoids copying large structs while guaranteeing no mutation.

```
void Modify(ref int x)    { x++; }
void TryGet(out int result){ result = 42; }
void ReadOnly(in int x)   { /* x++; compile error */ }

int a = 5; Modify(ref a);
int b;    TryGet(out b);
int c = 10; ReadOnly(in c);
```

Q9 Value Type vs Reference Type

- Value types (struct, enum, primitives) hold the actual data; reference types (class, array, string, delegate) hold a reference to the data on the heap.
- Assigning a value type COPIES the value; assigning a reference type copies only the REFERENCE, both variables point to the same object.
- Default of a value type is a zero-equivalent (0, false); default of a reference type is null.

Q10 Stack vs Heap

- **Stack**: stores value types and method call frames; LIFO order; very fast allocation/deallocation; limited size (StackOverflowException if exceeded).
- **Heap**: stores reference-type objects; managed by the Garbage Collector; slower to allocate, but much larger and more flexible.
- Stack memory is reclaimed automatically the moment a method returns; heap memory is reclaimed later by the GC.

Q11 == vs .Equals()

- **==** is an operator that CAN be overloaded; by default on reference types it compares identity, but types like string override it for value comparison.
- **.Equals()** is a virtual method from Object; it can be overridden to provide custom value-equality logic (e.g. in records).
- For value types both usually compare values; best practice is to override Equals and == together when you need value semantics.

Q12 Equals() vs ReferenceEquals()

- **Equals()** can be overridden by a class to define custom equality logic.
- **ReferenceEquals()** is static and always checks true object identity (same memory address) — it can NOT be overridden.
- Useful to detect whether Equals was overridden: compare Equals() result against ReferenceEquals() result.

Q13 String vs StringBuilder

- **string** is immutable — every modification creates a brand-new string object in memory.
- **StringBuilder** is mutable — it edits an internal buffer in place, which is far more efficient for repeated changes.
- Use plain string for a handful of simple operations; use StringBuilder inside loops or heavy concatenation scenarios.

```
string s = "Hello";
s += " World";           // creates a NEW string each time

var sb = new StringBuilder("Hello");
sb.Append(" World");     // modifies the SAME buffer
```

Q14 String vs StringBuilder vs StringBuffer

- **StringBuffer does not exist in C#/I.NET** — it is a Java class, often confused with StringBuilder in interviews.
- In Java: StringBuffer is thread-safe (synchronised); Java's StringBuilder is not.
- In C#, StringBuilder is NOT synchronised/thread-safe by default — you need external locking for concurrent access.
- So in .NET the real comparison is simply String vs StringBuilder (see Q13); 'StringBuffer' is a cross-language trick question.

Q15 var vs dynamic vs object

- **var**: type inferred by the COMPILER; still fully statically typed; must be initialised at the point of declaration.
- **dynamic**: type resolved at RUNTIME; bypasses compile-time checking; the underlying type can even change later.
- **object**: the base type of everything; statically typed, but needs an explicit CAST before you can use the actual type's members.

```
var a = 10;           // compiler infers int, checked at compile time
dynamic b = 10;
b = "now a string";   // legal – type resolved at runtime
object c = 10;
int n = (int)c;       // needs an explicit cast
```

Q16 is vs as

- **is** checks type-compatibility and returns a bool (can also pattern-match into a variable: `if (obj is string s)`).
- **as** attempts a cast and returns null if it fails (only for reference/nullable types) — no exception is thrown.
- A direct cast `(Type)obj` throws `InvalidCastException` on failure; `as` is the safer option when the cast isn't guaranteed.

Q17 Method Overloading vs Method Overriding

- **Overloading**: same method name, different parameter list, defined in the SAME class — resolved at compile time (early binding).
- **Overriding**: a derived class re-implements a base class's virtual/abstract method with the SAME signature — resolved at run time (late binding), using virtual/override.
- Overloading = compile-time polymorphism; Overriding = run-time polymorphism.

Q18 Virtual vs Override vs New

- **virtual**: marks a base class method as safe to be overridden.
- **override**: supplies a new implementation in the derived class and takes part in polymorphism (base-type reference calls the derived version).
- **new** (method hiding): hides the base member — calling through a base-type reference still runs the BASE version, no polymorphism.

```
class Base    { public virtual void Show() => Console.WriteLine("Base"); }
class Derived : Base { public override void Show() => Console.WriteLine("Derived (override)"); }
class Derived2 : Base { public new void Show()      => Console.WriteLine("Derived2 (new)"); }

Base b1 = new Derived(); b1.Show(); // "Derived (override)"
Base b2 = new Derived2(); b2.Show(); // "Base" (hidden, not polymorphic)
```

Q19 Static Class vs Singleton

- **Static class**: cannot be instantiated; ALL members must be static; no instance state; cannot implement interfaces or be inherited.
- **Singleton**: an ordinary class with a private constructor and one controlled instance exposed via a static property — it CAN implement interfaces, hold instance state, and be passed around as a parameter.
- Singleton supports lazy initialisation and can be swapped/mockd in unit tests via its interface; a static class cannot.

Q20 Early Binding vs Late Binding

- **Early (static) binding**: the method call is resolved at COMPILE time (normal calls, overload resolution, non-virtual methods).
- **Late (dynamic) binding**: the method call is resolved at RUN time (virtual/override calls through polymorphism, reflection, the dynamic keyword).
- Early binding is faster since it's checked upfront; late binding trades some performance for flexibility.

Q21 Encapsulation vs Abstraction

- **Encapsulation:** bundling data and behaviour together and hiding internal state using access modifiers (private fields + public properties/methods) — it hides the **'how'**.
- **Abstraction:** hiding complexity by exposing only the essential features through abstract classes/interfaces — it hides **implementation detail** from the consumer.
- Encapsulation is a data-hiding mechanism; abstraction is a design-level concept about what to reveal.

Q22 Inheritance vs Composition

- **Inheritance:** an 'is-a' relationship — a derived class extends a base class; results in tighter coupling.
- **Composition:** a 'has-a' relationship — a class contains instances of other classes to reuse their functionality; looser coupling, more flexible.
- The common design principle 'favour composition over inheritance' exists because composition avoids the fragile base-class problem.

Q23 Interface vs Abstract Class (when to choose)

- Choose an **interface** when unrelated classes need to share a common capability/contract (e.g. `Comparable`, `ILogger`).
- Choose an **abstract class** when related classes share common state and some common implementation, and you want a shared base.
- A class can implement many interfaces but extend only one abstract class — interfaces model 'can-do', abstract classes model 'is-a'.

Q24 Association vs Aggregation vs Composition

- **Association:** a general relationship between two independent classes (e.g. Teacher and Student) — both can exist on their own.
- **Aggregation:** a weak 'has-a' relationship — the whole and the part can exist independently (a Department has Teachers, but a Teacher can exist without a Department).
- **Composition:** a strong 'has-a' relationship — the part's lifecycle is bound to the whole (a House has Rooms; delete the House and its Rooms go too).

Q25 Shallow Copy vs Deep Copy

- **Shallow copy**: copies top-level fields only; if a field is a reference type, only the reference is copied — both objects still point to the SAME nested object. Achieved via `MemberwiseClone()`.
- **Deep copy**: recursively copies every nested object too, so the copy is fully independent of the original. Usually requires manual code or serialization-based cloning.

```
class Address { public string City; }
class Person : ICloneable
{
    public string Name;
    public Address Addr;

    public object Clone() => MemberwiseClone();    // shallow copy

    public Person DeepClone() => new Person
    {
        Name = this.Name,
        Addr = new Address { City = this.Addr.City } // independent copy
    };
}
```

Q26 Managed Code vs Unmanaged Code

- **Managed code** runs under the CLR — it benefits from garbage collection, type safety and structured exception handling (e.g. C#, VB.NET).
- **Unmanaged code** runs directly on the OS without the CLR — memory is managed manually (e.g. C, C++).
- .NET can interoperate with unmanaged code through P/Invoke or COM Interop.

Q27 Garbage Collection vs IDisposable

- **Garbage Collection** automatically reclaims memory of managed objects once nothing references them — timing is non-deterministic.
- **IDisposable** is a pattern for DETERMINISTIC cleanup of unmanaged resources (files, DB connections, sockets) via `Dispose()`, typically invoked with `using``.
- The GC only knows about managed memory; unmanaged handles must be released explicitly via `Dispose`, or they leak.

```
using (var conn = new SqlConnection(connStr))
{
    conn.Open();
} // Dispose() is called automatically here
```

Q28 Dispose vs Finalize

- **Dispose()**: explicit and deterministic — called directly by the developer (or via `using``); releases both managed and unmanaged resources immediately.
- **Finalize()** (destructor `~Class()`): called by the GC non-deterministically right before the object is collected — acts as a safety net for unmanaged cleanup.
- Best practice: implement the full `Dispose` pattern and call `GC.SuppressFinalize(this)` once `Dispose` has already run, so the finalizer isn't needed.

Q29 Exception vs Error

- **Exception**: a runtime problem that CAN be caught and handled gracefully (e.g. `DivideByZeroException`, `NullReferenceException`).
- **Error**: usually refers to a serious, generally UNRECOVERABLE condition the app should not try to handle (e.g. `StackOverflowException`, `OutOfMemoryException`).
- Rule of thumb: exceptions are expected/recoverable failure conditions; errors are severe, system-level issues.

Q30 Throw vs Throw ex

- **throw;** (bare) re-throws the current exception while **PRESERVING** the original stack trace.
- **throw ex;** re-throws but **RESETS** the stack trace to the current line, hiding where the exception actually originated.
- Always prefer bare `throw;` inside a catch block when you need to re-throw.

```
try { DoWork(); }
catch (Exception ex)
{
    LogError(ex);
    throw;           // preserves original stack trace ✓
    // throw ex;     // resets the stack trace ✗
}
```

Q31 Delegate vs Event

- **Delegate:** a type-safe function pointer; ANY code holding a reference to it can invoke it directly; supports multicast.
- **Event:** a controlled wrapper around a delegate that only lets external code subscribe (`+=`) or unsubscribe (`-=`) — it can't be invoked (fired) from outside the declaring class.
- Events enforce the publisher/subscriber pattern; raw delegates don't restrict who can trigger them.

```
public delegate void Notify(string msg);

public class Publisher
{
    public event Notify OnChanged;           // event - safely encapsulated
    public Notify RawDelegate;              // delegate - anyone can invoke

    public void Raise() => OnChanged?.Invoke("changed");
}
```

Q32 Delegate vs Func vs Action

- **Delegate**: a custom type you define yourself with the `delegate`` keyword — full control over the signature and name.
- **Func<T...>**: a built-in generic delegate that RETURNS a value (the last type parameter is the return type).
- **Action<T...>**: a built-in generic delegate that returns void.
- There's also **Predicate<T>**, a built-in delegate that always returns bool.

```
Func<int, int, int> add = (a, b) => a + b;           // returns int
Action<string> log = msg => Console.WriteLine(msg); // returns void
```

Q33 IEnumerable vs ICollection vs IList

- **IEnumerable<T>**: read-only, forward-only iteration (foreach); no Count, no indexing; supports deferred (lazy) LINQ execution.
- **ICollection<T>**: extends IEnumerable, adds Count, Add, Remove, Contains — knows its size and can be modified.
- **IList<T>**: extends ICollection, adds index-based access (`this[int]`), Insert and RemoveAt.

Capability	IEnumerable	ICollection	IList
Iterate (foreach)	Yes	Yes	Yes
Count property	No	Yes	Yes
Add / Remove	No	Yes	Yes
Index access [i]	No	No	Yes

Q34 IEnumerable vs IQueryable

- **IEnumerable**: executes the query IN MEMORY (LINQ to Objects) — good once a collection is already loaded.
- **IQueryable**: builds an EXPRESSION TREE and executes it at the data source (e.g. translated into SQL for a database), so filtering happens where the data lives.
- Use IQueryable for remote data sources like EF Core queries; switch to IEnumerable only after materialising the results with `ToList()`.

Q35 List<T> vs Array

- **Array**: fixed size once created — it can never grow or shrink.
- **List<T>**: dynamically resizable, internally backed by an array that grows automatically; provides Add/Remove/Insert helpers.
- Array can have marginally less overhead for fixed-size scenarios; List is far more convenient for collections whose size changes.

Q36 Dictionary vs Hashtable

- **Dictionary<TKey,TValue>**: generic, type-safe, no boxing overhead for value types — introduced in .NET 2.0.
- **Hashtable**: non-generic, stores everything as `object`` (boxing/unboxing overhead) — a legacy .NET 1.x type.
- Dictionary throws an exception when you access a missing key via the indexer; Hashtable simply returns null.
- Recommendation: always prefer Dictionary<TKey,TValue> in modern code.

Q37 Thread vs Task

- **Thread**: represents an actual OS-level thread — low-level, heavier, managed directly by you.
- **Task**: a higher-level abstraction over the ThreadPool, representing an asynchronous operation with results, continuations, cancellation and exception handling built in.
- Task is generally preferred for modern async workloads — it scales far better thanks to thread-pool reuse.

Q38 Task vs async/await

- **Task / Task<T>**: represents the actual asynchronous operation and its eventual result — the 'what'.
- **async/await**: language keywords used to WORK WITH Tasks in a readable, sequential-looking way — the 'how'.
- `async`` marks a method so it can use `await``; `await`` suspends execution until the awaited Task completes, WITHOUT blocking the calling thread.

```
public async Task<string> GetDataAsync()
{
    var result = await httpClient.GetStringAsync(url); // non-blocking wait
    return result;
}
```

Q39 Parallelism vs Concurrency

- **Concurrency**: multiple tasks progress in an interleaved/overlapping manner (can even happen on a single core) — about structuring work.
- **Parallelism**: multiple tasks execute literally AT THE SAME TIME on multiple cores/processors — about simultaneous execution.
- Concurrency can exist without parallelism (single-core time-slicing); parallelism always requires multiple execution units.

Q40 Synchronous vs Asynchronous Programming

- **Synchronous**: statements execute one after another; each one blocks and waits for the previous to finish.
- **Asynchronous**: operations can run independently — the calling thread is freed up while waiting (typically for I/O), improving responsiveness and scalability.
- Async shines for I/O-bound work (DB calls, HTTP requests, file access) where threads would otherwise just sit idle waiting.

03

ASP.NET Core

Web framework internals, security, filters, DI lifetimes and HTTP semantics.

Q41 ASP.NET MVC vs ASP.NET Core

- **ASP.NET MVC**: part of the older .NET Framework — Windows-only and IIS-dependent.
- **ASP.NET Core**: cross-platform (Windows/Linux/macOS), open-source, modular via NuGet, faster, ships with built-in Dependency Injection, and can be hosted on Kestrel, IIS or Nginx.
- ASP.NET Core merges MVC and Web API into ONE unified framework — the same controllers can serve views and APIs.

Q42 MVC vs Web API

- **MVC**: designed to return Views (HTML) to browsers — typically inherits from `Controller`, returns `ViewResult`.
- **Web API**: designed to return data (JSON/XML) to clients like SPAs and mobile apps — typically inherits from `ControllerBase` with `[ApiController]`.
- In ASP.NET Core the two are unified; the real difference today is which base class and attributes you choose.

Q43 Controller vs API Controller

- **Controller** (inherits `Controller``): supports both Views and API responses; provides `ViewBag/ViewData/View()` helpers.
- **API Controller** (inherits `ControllerBase`` + `[ApiController]`): lighter weight, no view support, automatic model-state validation and automatic 400 responses, binding source inference.
- Use `ControllerBase` + `[ApiController]` for pure REST APIs; use `Controller` when Razor Views are also needed.

Q44 Middleware vs Filter

- **Middleware**: sits in the HTTP PIPELINE and processes every request/response regardless of controller (logging, auth, CORS, exception handling) — configured in `Program.cs`.
- **Filter**: sits inside the MVC action-invocation pipeline, tied to controller/action execution (Authorization, Action, Result, Exception filters) — has access to MVC-specific context like route data and model state.
- Middleware runs earlier and more globally; filters run within the MVC layer.

Q45 Authentication vs Authorization

- **Authentication**: verifying WHO the user is (e.g. validating a login or a JWT token).
- **Authorization**: verifying WHAT the authenticated user is allowed to do (e.g. `[Authorize(Roles="Admin")]`).
- Authentication always happens first; authorization decisions depend on the authenticated identity.

Q46 JWT vs Cookie Authentication

- **JWT**: stateless — a signed token holding claims, stored client-side and sent in the Authorization header; great for APIs/SPAs/mobile, scales well since no server-side session is required.
- **Cookie Authentication**: the server issues a cookie (often an encrypted session id), the browser sends it automatically; stateful, well suited to traditional server-rendered web apps.
- JWTs are harder to revoke instantly (until they expire); cookies can be invalidated server-side immediately.

Q47 Session vs Cookies

- **Cookie**: a small piece of data stored on the CLIENT, sent automatically with every request to the same domain.
- **Session**: data stored on the SERVER, identified by a session id which is itself usually stored inside a cookie.
- Cookies are size-limited (~4KB) and visible/editable client-side; session data stays server-side and is generally more secure.

Q48 ViewData vs ViewBag vs TempData

- **ViewData**: a dictionary, key-value based, needs casting on read, lives only for the current request.
- **ViewBag**: a dynamic wrapper over ViewData — no casting needed, same lifetime as ViewData (current request only).
- **TempData**: persists across ONE additional redirect (backed by session or cookies) — perfect for passing a success message after a redirect.

Q49 Action Filter vs Authorization Filter

- **Authorization Filter**: runs FIRST, before model binding and any other filters — checks whether the user may access the resource, and can short-circuit the pipeline.
- **Action Filter**: runs AFTER authorization and model binding, wrapping the action method itself — used for logging, modifying arguments/results, validation.
- Pipeline order: Authorization → Resource → Model Binding → Action → Exception → Result filters.

Q50 Singleton vs Scoped vs Transient

- **Singleton**: ONE instance for the entire application lifetime, shared everywhere.
- **Scoped**: ONE instance PER HTTP request (or per scope) — shared within that request, but different across requests.
- **Transient**: a BRAND-NEW instance is created every single time it's requested, even within the same request.

```
builder.Services.AddSingleton<IMyService, MyService>();  
builder.Services.AddScoped<IMyService, MyService>();  
builder.Services.AddTransient<IMyService, MyService>();
```

Q51 Dependency Injection vs Service Locator

- **Dependency Injection**: dependencies are PUSHED into a class from the outside (constructor/property) — the class never creates them itself; promotes loose coupling and testability.
- **Service Locator**: the class PULLS its dependencies from a central registry at runtime (`container.Resolve()`) — hides the real dependency list and is harder to test; considered an anti-pattern by many.
- Modern ASP.NET Core apps favour built-in DI over the Service Locator pattern.

Q52 Constructor Injection vs Method Injection

- **Constructor Injection:** dependencies are supplied through the constructor and available for the WHOLE lifetime of the object — the most common, recommended approach.
- **Method Injection:** a dependency is passed as a parameter to ONE specific method, used when it's only needed for that single operation, not the entire class.

Q53 IActionResult vs ActionResult vs ActionResult<T>

- **IActionResult:** an interface; used when you may return DIFFERENT result types (Ok(), NotFound(), BadRequest()) but not a plain object directly.
- **ActionResult:** a concrete class implementing IActionResult, used similarly.
- **ActionResult<T>:** lets you return EITHER a concrete type T directly OR any IActionResult (like NotFound()) — recommended for Web API actions since .NET Core 2.1.

```
[HttpGet("{id}")]
public ActionResult<Product> Get(int id)
{
    var p = _repo.Find(id);
    if (p == null) return NotFound(); // IActionResult path
    return p;                       // Product (T) path
}
```

Q54 HTTP GET vs POST

- **GET:** retrieves data, parameters travel in the URL (query string); idempotent, safe, cacheable; has length limits; shouldn't cause side effects.
- **POST:** sends data in the request BODY, used to create/submit data; NOT idempotent — calling it twice can create two resources; not cached by default.

Q55 PUT vs PATCH

- **PUT:** replaces the ENTIRE resource with the supplied representation; idempotent — repeating it has the same effect.
- **PATCH:** applies a PARTIAL update — only the specified fields change; idempotency depends on the implementation.

Q56 POST vs PUT

- **POST**: used to CREATE a new resource; NOT idempotent — repeated identical POSTs can create multiple resources.
- **PUT**: used to CREATE or REPLACE a resource at a known URI; idempotent — repeated identical PUTs leave the same final state.
- Rule of thumb: `POST /orders` (server assigns the id) vs `PUT /orders/5` (client specifies the exact resource).

Q57 HTTP 401 vs 403

- **401 Unauthorized**: the request is missing valid AUTHENTICATION credentials — 'who are you?'.
- **403 Forbidden**: the server knows exactly who you are, but you don't have PERMISSION to access this resource — 'I know you, but you can't do this'.

Q58 HTTP 404 vs 400

- **400 Bad Request**: the request itself is malformed or invalid (bad JSON, missing required field) — a client-side error before real processing even starts.
- **404 Not Found**: the request was well-formed, but the requested resource simply doesn't exist at that URL.

Q59 HTTP 500 vs 503

- **500 Internal Server Error**: a generic failure — something went wrong on the SERVER (unhandled exception/bug) while processing an otherwise valid request.
- **503 Service Unavailable**: the server is temporarily UNABLE to handle the request (overloaded, in maintenance, or a dependency is down) — often includes a Retry-After header.

Q60 REST API vs SOAP API

- **REST**: an architectural style using standard HTTP verbs and lightweight formats (usually JSON); stateless and easy to consume — dominant for web/mobile.
- **SOAP**: a strict PROTOCOL using XML exclusively, with a formal contract (WSDL) and built-in standards for security/error handling (WS-Security) — heavier, common in enterprise/legacy systems (banking, telecom).

04

Entity Framework Core

ORM concepts, query execution, loading strategies and change tracking.

Q61 EF Core vs ADO.NET

- **ADO.NET:** low-level data access — you write raw SQL and manually manage connections/commands/readers; full control and best raw performance.
- **EF Core:** an ORM built on top of ADO.NET; you work with C# objects and LINQ instead of SQL, and it handles change tracking and migrations automatically — more productive, but with some performance overhead.

Q62 EF Core vs Dapper

- **EF Core:** full-featured ORM with change tracking, migrations, eager/lazy loading, and LINQ-to-SQL translation — more abstraction, faster development.
- **Dapper:** a lightweight 'micro-ORM' — maps query results to objects but you write the raw SQL yourself; very close to ADO.NET performance, no built-in change tracking or migrations.
- Choose EF Core for productivity on complex domain models; choose Dapper for performance-critical, simpler data access.

Q63 Code First vs Database First

- **Code First:** you define C# entity classes first, and EF Core generates/updates the database schema via migrations — ideal for new, model-driven projects.
- **Database First:** you start with an EXISTING database and EF Core scaffolds entity classes and a DbContext from it — ideal when a legacy or DBA-managed database already exists.

Q64 Tracking vs NoTracking

- **Tracking** (default): EF Core keeps a snapshot of each entity in its ChangeTracker so it can detect changes for SaveChanges() — needed when you plan to UPDATE the entity.
- **NoTracking** (`.AsNoTracking()`): entities are NOT tracked — faster, uses less memory, ideal for read-only queries such as GET endpoints.

```
var products = await _context.Products
    .AsNoTracking()
    .Where(p => p.IsActive)
    .ToListAsync();           // read-only, faster
```

Q65 Find() vs FirstOrDefault()

- **Find()**: looks up an entity by its PRIMARY KEY; checks the local ChangeTracker cache FIRST, avoiding a DB round-trip if it's already tracked.
- **FirstOrDefault()**: runs a LINQ query with any condition — always hits the database, but supports any predicate, not just the primary key.
- Find() is typically faster for a simple primary-key lookup on an entity that may already be tracked in memory.

Q66 First() vs FirstOrDefault()

- **First()**: throws an InvalidOperationException if the sequence is empty / nothing matches.
- **FirstOrDefault()**: returns the type's default value (null, 0, etc.) instead of throwing when there is no match.
- Use First() when a result is guaranteed; use FirstOrDefault() when 'no match' is a valid, expected outcome.

Q67 Single() vs SingleOrDefault()

- **Single()**: expects EXACTLY one match — throws if there are zero OR more than one matching elements.
- **SingleOrDefault()**: returns the default value on zero matches, but still throws if there is MORE than one match.
- Use these to enforce/verify uniqueness, e.g. looking up a record by a unique key like Email.

Q68 Where() vs Find()

- **Where()**: a general-purpose LINQ filter that accepts any predicate; always queries the database; can return zero, one, or many rows.
- **Find()**: specifically looks up a SINGLE entity by its primary key, and can skip the database round-trip if it's already tracked.

Q69 Eager Loading vs Lazy Loading vs Explicit Loading

- **Eager Loading**: related data is loaded UPFRONT, in the same query, via `.Include()` — avoids the N+1 query problem.
- **Lazy Loading**: related data loads AUTOMATICALLY the moment a navigation property is first accessed — requires virtual navigation properties, and can silently trigger many extra queries (N+1).
- **Explicit Loading**: related data is loaded MANUALLY on demand via `.Entry(entity).Reference().Collection().Load()`.

```
// Eager
var orders = context.Orders.Include(o => o.Customer).ToList();

// Explicit
var order = context.Orders.Find(1);
context.Entry(order).Reference(o => o.Customer).Load();
```

Q70 IEnumerable vs IQueryable in EF Core

- Same core idea as Q34, but specifically in EF Core: `IQueryable` defers execution and translates the WHOLE query chain into SQL that runs on the database server.
- Switching to `IEnumerable` too early (e.g. `.ToList()` followed by `.Where()`) pulls ALL rows into memory first, then filters in C# — much less efficient.
- Always keep a query as `IQueryable` for as long as possible, and materialise it (`ToList/ToListAsync`) only at the very end.

Q71 Migration vs Database Update

- **Migration** (``dotnet ef migrations add``): generates a C# migration FILE describing the schema change based on your current model — it does NOT touch the database yet.
- **Database Update** (``dotnet ef database update``): actually APPLIES the pending migration(s) to the real database.
- Typical workflow: change the model → add a migration → review the generated code → update the database.

Q72 DbContext vs DbSet

- **DbContext**: represents a SESSION with the database — manages the connection, change tracking, and exposes `SaveChanges()`; the entry point to data access.
- **DbSet<TEntity>**: represents a collection/table for a specific entity type inside the `DbContext`, used to query and add/remove instances (e.g. `context.Products`).

Q73 Stored Procedure vs LINQ Query

- **Stored Procedure:** precompiled SQL stored in the database; can be faster for complex, repeated operations, but logic lives outside the app codebase.
- **LINQ Query:** written in C#, translated to SQL by EF Core at runtime; logic stays with the application, easier to maintain/debug/test, and more database-portable.
- Use stored procedures for heavy reporting/bulk operations managed by DBAs; LINQ for typical application CRUD logic.

Q74 SaveChanges vs SaveChangesAsync

- **SaveChanges():** SYNCHRONOUS — blocks the calling thread until the database write finishes.
- **SaveChangesAsync():** ASYNCHRONOUS — frees the thread while waiting for the I/O-bound database operation, improving scalability under concurrent load.
- Best practice: always prefer SaveChangesAsync() inside ASP.NET Core applications.

05

SQL Server

Database fundamentals, indexing, joins, keys and query design.

Q75 DELETE vs TRUNCATE vs DROP

- **DELETE:** removes rows based on a WHERE condition (or all rows); a DML operation, logged row-by-row, CAN be rolled back, and triggers fire.
- **TRUNCATE:** removes ALL rows instantly by deallocating data pages; minimally logged, much faster, resets the identity counter; cannot use WHERE, and triggers do not fire.
- **DROP:** removes the ENTIRE table — structure, data, indexes and constraints — completely from the database.

Q76 WHERE vs HAVING

- **WHERE:** filters INDIVIDUAL ROWS BEFORE grouping happens; cannot directly use aggregate functions like SUM or COUNT.
- **HAVING:** filters GROUPS AFTER GROUP BY has been applied, and CAN use aggregate functions in its condition.

```
SELECT DeptId, COUNT(*) AS EmpCount
FROM Employees
WHERE Status = 'Active'      -- filters rows first
GROUP BY DeptId
HAVING COUNT(*) > 5;        -- filters groups after
```

SQL

Q77 INNER JOIN vs LEFT JOIN

- **INNER JOIN:** returns ONLY the rows that have matching values in BOTH tables.
- **LEFT JOIN (LEFT OUTER JOIN):** returns ALL rows from the left table, plus matching rows from the right table; unmatched right-side columns come back as NULL.

```
SELECT e.Name, d.DeptName
FROM Employees e
LEFT JOIN Departments d ON e.DeptId = d.Id;
-- returns every employee, even those with no department
```

SQL

Q78 UNION vs UNION ALL

- **UNION:** combines two or more result sets and REMOVES duplicate rows — requires an internal sort/distinct step, so it's slower.
- **UNION ALL:** combines the result sets and KEEPS all rows, including duplicates — faster, since no de-duplication is needed.
- Both require the combined queries to have the same number of columns with compatible data types.

Q79 Primary Key vs Unique Key

- **Primary Key:** uniquely identifies every row, does NOT allow NULLs, and only ONE is allowed per table (can be composite).
- **Unique Key:** also enforces uniqueness, but allows ONE NULL value per column, and a table CAN have multiple unique keys.
- By default, a primary key creates a CLUSTERED index; a unique key creates a NON-CLUSTERED index.

Q80 Primary Key vs Foreign Key

- **Primary Key:** uniquely identifies each row WITHIN its own table.
- **Foreign Key:** a column (or set of columns) in one table that REFERENCES the primary key of another (or the same) table, enforcing referential integrity.
- Foreign key values can repeat and can be NULL (unless restricted); primary key values must always be unique and non-null.

Q81 Clustered vs Non-Clustered Index

- **Clustered Index:** determines the PHYSICAL ORDER of data rows in the table — the data IS the index; only ONE allowed per table (usually the primary key).
- **Non-Clustered Index:** a SEPARATE structure holding a sorted pointer to the actual data rows — a table can have MANY non-clustered indexes.
- Clustered lookups are generally faster since the data is physically sorted; non-clustered lookups need an extra step to fetch the row unless it's a covering index.

Q82 CHAR vs VARCHAR

- **CHAR(n):** FIXED-length storage — always uses n characters, padding shorter values with spaces.
- **VARCHAR(n):** VARIABLE-length storage — uses only as much space as needed, up to the maximum n.
- Use CHAR for fixed-length codes (e.g. 'US', 'IN'); use VARCHAR for naturally variable-length text such as names.

Q83 VARCHAR vs NVARCHAR

- **VARCHAR:** stores non-Unicode (ASCII) data — one byte per character, more storage-efficient.
- **NVARCHAR:** stores UNICODE data, supporting multiple languages/scripts (Chinese, Arabic, emoji) — typically two bytes per character, so it uses more storage.
- Use NVARCHAR when multilingual/international text must be supported; use VARCHAR when data is guaranteed to be ASCII, to save space.

Q84 Stored Procedure vs Function

- **Stored Procedure:** can perform DML (INSERT/UPDATE/DELETE), can return zero, one, or many result sets/output parameters, cannot be used inside a SELECT, and can manage transactions.
- **Function:** MUST return a value (scalar or table), generally cannot modify database data, but CAN be used directly inside a SELECT/WHERE clause.

Q85 View vs Stored Procedure

- **View**: a virtual table based on a stored SELECT query — queried like a normal table; mainly used to simplify complex queries or restrict row/column access; cannot accept parameters or hold procedural logic.
- **Stored Procedure**: a set of precompiled, procedural SQL statements that CAN accept parameters, perform DML, use loops/conditions, and return result sets — much more flexible than a view.

Q86 CTE vs Temporary Table

- **CTE** (`WITH ... AS`): a temporary, named result set that exists only for the duration of a SINGLE query — not physically stored, improves readability, and supports recursion.
- **Temporary Table** (`#Temp`): a real table stored in TempDB — persists across MULTIPLE statements in the session, can have indexes and constraints, and works better for larger, reusable datasets.

Q87 Temporary Table vs Table Variable

- **Temporary Table** (`#Temp`): stored in TempDB, supports indexes/constraints, statistics are maintained — better suited for LARGE datasets and complex queries.
- **Table Variable** (`@TableVar`): also backed by TempDB, but scoped only to the batch/procedure where it's declared, and historically has no statistics — better for SMALL, simple datasets.

Q88 DELETE vs MERGE

- **DELETE**: a single DML statement that removes rows matching a condition from ONE table.
- **MERGE**: a statement that performs INSERT, UPDATE and DELETE all in ONE statement, comparing a source and target table — useful for synchronising (upserting) two tables at once.

Q89 Normalization vs Denormalization

- **Normalization**: organising data into multiple related tables to eliminate redundancy and preserve data integrity (1NF, 2NF, 3NF...) — better suited for write-heavy, transactional (OLTP) systems.
- **Denormalization**: intentionally introducing some redundancy (combining tables, duplicating data) to reduce joins and speed up READS — better suited for read-heavy reporting/analytics (OLAP) systems.

Q90 Subquery vs CTE

- **Subquery**: a query NESTED inside another query (in SELECT/WHERE/FROM); can get hard to read when deeply nested and is NOT reusable within the same query.
- **CTE**: a named, temporary result set defined ONCE at the top of the query, which CAN be referenced multiple times in the same query — improves readability and uniquely supports recursion.